
SISTEMA DE DRONES PARA ENVIAR MENSAJES SEGUROS

202408977 – Jonathan Eduardo Fuentes García

Resumen

Este proyecto consiste en un programa que controla varios drones para enviar mensajes en código. Cada dron puede subir o bajar y encender una luz. Dependiendo del dron y la altura a la que encienda la luz, se representa una letra. El programa recibe un archivo XML con la configuración de los drones, las tablas de letras y los mensajes que se quieren enviar. Luego, el sistema simula el movimiento de los drones segundo a segundo para encontrar el menor tiempo posible para enviar el mensaje. Se usaron listas enlazadas hechas a mano (sin usar las listas de C#) y se generaron reportes con Graphviz.

También se hizo una página web para que el usuario pueda cargar archivos, ver los drones, los sistemas y los mensajes, y ver gráficos de cómo se mueven los drones. El proyecto cumple con todo lo que pidió el profesor y funciona bien.

Palabras clave: Drones, simulación, tiempo óptimo, listas enlazadas, XML, página web.

Abstract

This project is a program that controls several drones to send coded messages. Each drone can go up or down and turn on a light. Depending on the drone and the height at which it turns on the light, a letter is represented. The program receives an XML file with the drone configuration, letter tables, and the messages to be sent. Then, the system simulates the movement of the drones second by second to find the shortest possible time to send the message. We used linked lists made by hand (without using C# lists) and generated reports with Graphviz.

We also made a web page so the user can upload files, view drones, systems, and messages, and see graphs of how the drones move. The project meets all the teacher's requirements and works well.

Keywords

Drones, simulation, optimal time, linked lists, XML, web page..

Introducción

En este proyecto se nos pidió hacer un programa que controle un montón de drones. La idea es que los drones puedan subir, bajar y encender una luz. Cada vez que un dron enciende la luz a cierta altura, eso representa una letra. Así podemos enviar mensajes en secreto. El gobierno crea una tabla donde dice qué letra es cada combinación de dron y altura. Nosotros tenemos que hacer un programa que, dado un mensaje, calcule la mejor forma de mover los drones para que enciendan las luces en el orden correcto y en el menor tiempo posible. También hay que hacer una página web para que sea fácil de usar. En este ensayo voy a explicar cómo hice el programa, qué problemas tuve y cómo los resolví.

Desarrollo del tema

a. Cómo guardé los datos (listas enlazadas)

Una regla importante era que no podíamos usar las listas normales de C# (como List o Queue). Teníamos que hacer nuestras propias listas usando nodos y punteros. Así que hice clases como NodoDron, ListaDrones, NodoMensaje, Lista Mensajes, etc.

Cada nodo guarda un dato (por ejemplo un dron) y una flecha que apunta al siguiente nodo. De esta manera puedo agregar, buscar y recorrer los elementos sin usar las librerías de C#. Esto me ayudó a entender mejor cómo funcionan las listas enlazadas y la memoria dinámica. Al principio fue difícil, pero después le agarré la mano.

b. La simulación del tiempo óptimo

Lo más complicado fue hacer que el programa calcule el menor tiempo posible para enviar un mensaje. La simulación funciona así: cada dron empieza en altura 1. El mensaje es una lista de instrucciones (por ejemplo: "el DronA debe prender la luz a 8 metros", luego "el DronB a 6 metros", etc.). El programa avanza segundo a segundo. En cada segundo, todos los drones se mueven un metro (suben o bajan) si necesitan llegar a su próxima altura. Solo un dron puede prender la luz por segundo. Además, los drones se preparan con anticipación: si un dron tiene que prender más adelante, ya va subiendo o bajando hacia esa altura mientras otros drones hacen lo suyo. Así se logra que el tiempo total sea el mínimo. Por ejemplo, si un mensaje tiene cuatro letras y cada letra usa un dron diferente, el tiempo puede ser muy corto porque los drones se mueven al mismo tiempo. En cambio, si todas las letras usan el mismo dron, el tiempo es la suma de los movimientos. Mi programa hace esto muy bien y los tiempos que calcula coinciden con los que yo sacaba a mano. Me costó un poco depurar errores, pero al final funcionó.

c. Los archivos XML

El profesor nos dio un formato especial para los archivos de entrada y salida. El archivo de entrada (entrada.xml) tiene la lista de drones, los sistemas de drones (cada sistema tiene su tabla de letras) y los mensajes con sus instrucciones.

Mi programa lee ese archivo y guarda toda la información en mis listas enlazadas. Luego, cuando el usuario quiere, puede generar un archivo de salida (salida.xml) que muestra para cada mensaje el tiempo óptimo y las acciones que debe hacer cada dron en cada segundo. Ese archivo tiene una estructura muy específica que el profesor pidió. Para leer y escribir XML usé las librerías normales de C#

(XmlDocument) porque eso sí estaba permitido. Aprendí que los XML son muy útiles para guardar datos de forma ordenada.

d. La página web y los dibujos con Graphviz

También hice una página web usando [ASP.NET](#) Core MVC. La página tiene un menú donde puedes cargar el archivo XML, ver la lista de drones (ordenados alfabéticamente), agregar un nuevo dron, ver los sistemas de drones, ver los mensajes y generar el XML de salida. Para los gráficos usamos Graphviz. Por ejemplo, cuando ves los sistemas de drones, el programa genera un archivo .dot y luego lo convierte a una imagen PNG usando el programa dot que viene con Graphviz. Esa imagen se muestra en la página. Lo mismo pasa cuando ves las instrucciones de un mensaje: se dibuja un gráfico que muestra, segundo a segundo, qué está haciendo cada dron (subir, bajar, esperar o prender la luz). Esto es muy útil para entender cómo se mueven los drones. Me gustó hacer la página web porque así cualquiera puede usar el programa sin tener que abrir la consola.

e. Diagramas que hice

Para la documentación hice dos diagramas. El primero es un diagrama de clases (Figura 1). Muestra todas las clases que usé y cómo se relacionan: GestorDatos que guarda las listas de drones, sistemas y mensajes; SistemaDrones que tiene la tabla de letras; Mensajes que tiene las instrucciones; SimuladorTiempo que hace la simulación; etc. El segundo es un diagrama de actividades (Figura 2). Muestra paso a paso cómo se simula un mensaje: desde que se cargan las instrucciones hasta que se calcula el tiempo final. Estos diagramas los hice con una herramienta en línea y los pegué en el Word. Hacer diagramas me ayudó a organizar mejor mis ideas.

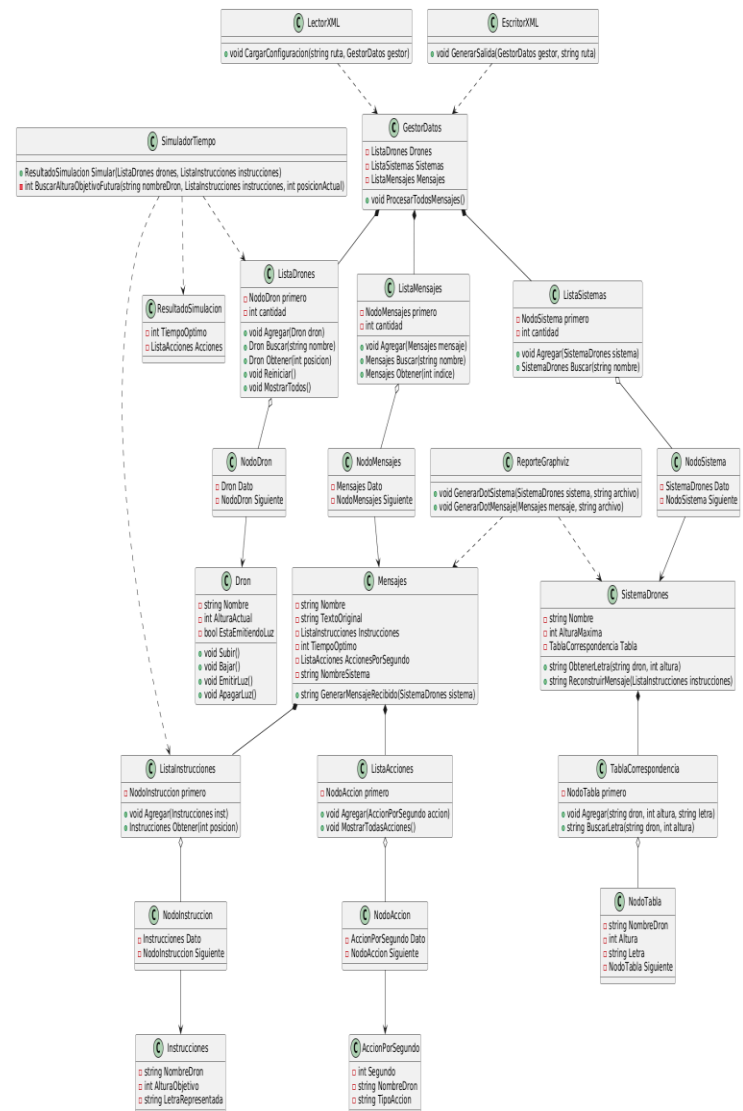


Figura 1. Diagrama de clases del programa.

Fuente: elaboración propia

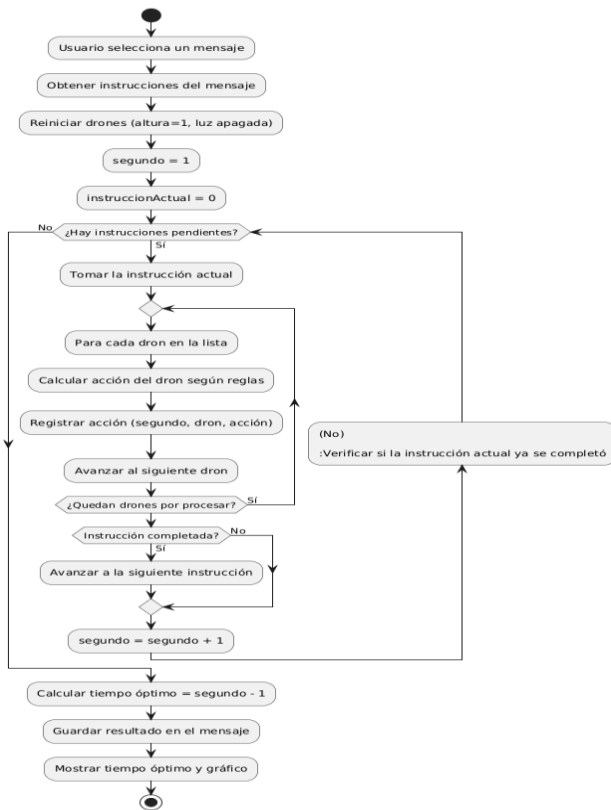


Figura 2. Diagrama de Actividades del programa.

Fuente: elaborado en PlantUML

Conclusiones

1. Logré hacer un programa completo que cumple todo lo que pedía el proyecto. Funciona bien y los tiempos que calcula son correctos.
2. Aprendí a hacer mis propias listas enlazadas sin usar las de C#, y eso me ayudó a entender mejor cómo funcionan los punteros y la memoria.

3. La simulación paralela de los drones es un buen ejemplo de cómo se pueden hacer varias cosas al mismo tiempo para ahorrar tiempo.
4. La página web quedó fácil de usar. Cualquier persona puede cargar un archivo, ver los datos y generar reportes sin tener que saber programar.
5. Si en el futuro quisiéramos agregar más reglas (por ejemplo, que los drones puedan moverse más de un metro por segundo o que puedan prender luces de diferentes colores), el programa se puede modificar sin mucho problema porque está bien organizado.

Referencias bibliográficas

- Microsoft. (2023). *Documentación de ASP.NET Core MVC*. Recuperado de <https://learn.microsoft.com>
- Graphviz. (s.f.). *Lenguaje DOT*. Recuperado de <https://graphviz.org/doc/info/lang.html>
- Universidad San Carlos de Guatemala. (2026). *Proyecto No. 2 – Introducción a la Programación y Computación 2*. Documento interno.
- Deitel, P., & Deitel, H. (2017). *C# 6 for Programmers*. Pearson.
- Fowler, M. (2003). *UML Distilled*. Addison-Wesley.

Gracias 😊